Topic: ADT of Stack, Operations on Stack, Array Implementation of Stack

Q. Define Stack as an Abstract Data Type (ADT). Explain its primary operations and demonstrate how a Stack can be implemented using an Array with suitable code snippets.

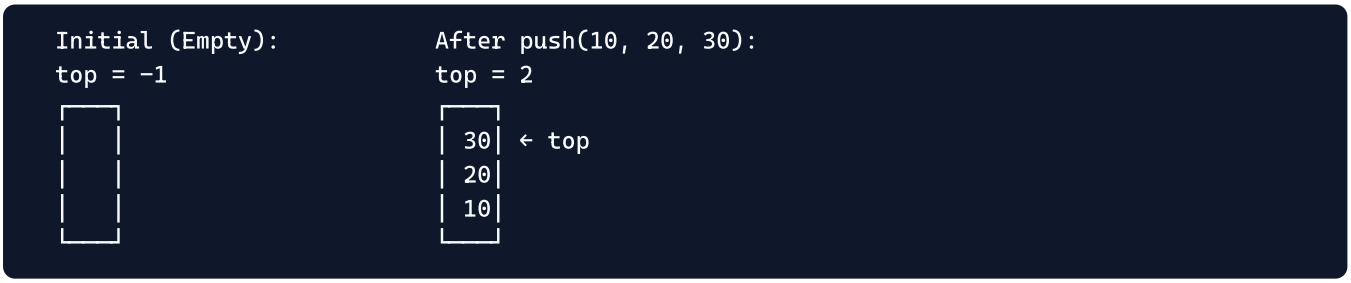
> 📌 **Definition to Remember**
> A **Stack** is a linear data structure that follows the **LIFO (Last In, First Out)** principle — the last element inserted is the first to be removed. All insertions and deletions occur at a single end called the **Top**. In an array implementation, `top = -1` represents an empty stack.

---

1. Stack as an ADT

Operation	Description	Condition
push(element)	Add element to the Top	**Stack Overflow** if full
pop()	Remove and return Top element	**Stack Underflow** if empty
peek() / top()	View top element without removing	Underflow if empty
isEmpty()	Returns True if stack has no elements	top == -1
isFull()	Returns True if stack is at max capacity	top == MAX-1

2. Stack Diagram



3. Array Implementation in C

```c
#include <stdio.h>
#define MAX 5

int stack[MAX];
int top = -1;

void push(int value) {
    if (top == MAX - 1)
        printf("Stack Overflow!\n");
    else
        stack[++top] = value;    // increment top, then insert
}

int pop() {
    if (top == -1) {
        printf("Stack Underflow!\n");
        return -1;
    }
    return stack[top--];         // return top element, then decrement
}

int peek() {
    if (top == -1) return -1;
    return stack[top];
}

int isEmpty() { return top == -1; }
```

4. Advantages and Limitations

	Array Implementation
Advantage	Simple, fast (O(1) for push/pop/peek), contiguous memory
Disadvantage	Fixed size — cannot grow dynamically (use Linked List for dynamic stack)

> ⭐ **Must-Write Points (for 10 marks)**
>
> 1. Stack follows **LIFO** (Last In, First Out) — all operations at the Top.
> 2. Five operations: **push, pop, peek, isEmpty, isFull**.
> 3. **Stack Overflow**: push on a full stack; **Stack Underflow**: pop from an empty stack.
> 4. Array implementation: `top` variable tracks the top index; `top = -1` means empty.
> 5. Push: `stack[++top] = value` ; Pop: `return stack[top--]` .
> 6. All stack operations (push/pop/peek) are **O(1)** time complexity.
> 7. Limitation of array implementation: **fixed size** — cannot grow dynamically at runtime.

> ⚡ **Quick Recall**
>
> `Stack → LIFO → Top pointer → push (Overflow) → pop (Underflow) → peek → isEmpty (top==-1) → Array: O(1), fixed size`